InferFlow: LLM Inference Routing Experiments

CS 6650 Final Project | Spring 2026 | AWS EKS · llama.cpp · Qwen2.5-0.5B-Instruct · Go Router

1. System Overview

InferFlow is a Go-based LLM inference router exposing an OpenAI-compatible `/v1/chat/completions` API in front of multiple llama.cpp backends. It implements four pluggable routing strategies switchable at runtime via `PUT /strategy`.

> **Infrastructure:** AWS EKS — 3 worker nodes (c5.xlarge, 4 vCPU / 8 GB each), each running llama.cpp serving *Qwen2.5-0.5B-Instruct* (GGUF, CPU-only). A Redis pod provides shared state. Router exposed via AWS ALB.

Strategy	Description
round_robin	Fixed-order cycle. Zero overhead.
random	Random backend selection each request.
least_pending	Routes to backend with fewest in-flight requests.
kv_aware	SHA-256 hashes prompt → Redis lookup for backend affinity; falls back to least-pending on miss; records result for next request.

2. Experiment 1 — Strategy Latency Comparison

Purpose & Tradeoff

How does each routing strategy perform under concurrent load? The tradeoff is **routing overhead vs. load balance quality**: simpler strategies add no overhead but cannot react to backend state; smarter strategies inspect state but add per-request cost.

Setup

- 400 requests total — 100 per strategy, concurrency 3, mixed prompts (every 3rd request repeats a fixed prompt)
- Strategy switched live via `PUT /strategy` between runs; latency measured end-to-end (client round-trip)

Results

Strategy	p50 (ms)	p95 (ms)	max (ms)	Backend spread
round_robin	4693	5296	7995	34 / 33 / 33
least_pending	**4757**	**4860**	**4995**	**33 / 33 / 34**
kv_aware	4812	6906	8930	46 / 30 / 24
random	5949	7789	9046	35 / 33 / 32

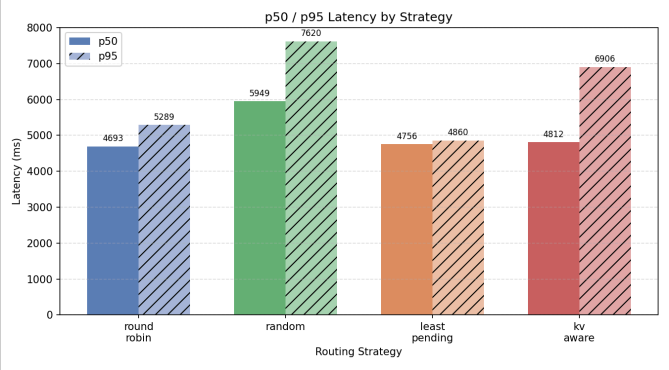

Fig 1 — p50/p95 latency per strategy

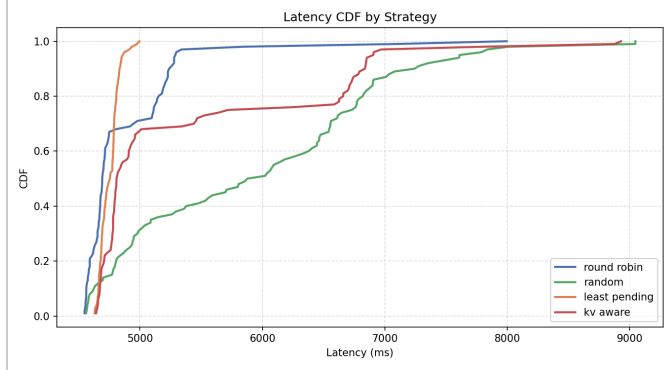

Fig 2 — CDF: random's long tail is clearly visible

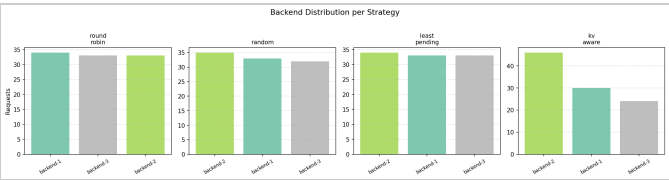

Fig 3 — Backend distribution: kv_aware hotspots on backend-2 (46%)

Fig 4 — Latency over time: random spikes; least_pending stays flat

Analysis

least_pending delivers the best tail latency — p95 of 4860ms and max of 4995ms, roughly 40% lower max than round_robin (7995ms) and 55% lower than random (9046ms). By routing to the least-loaded backend, it absorbs the natural variance in llama.cpp inference times.

round_robin achieves the lowest p50 (4693ms) but ignores backend state, so a slow backend continues receiving requests at the same rate, causing tail spikes.

random is the worst at every percentile. Its p50 (5949ms) is 1.3 seconds slower than round_robin — random backend collisions stack requests on a single backend while others sit idle.

kv_aware achieves 100% Redis cache hits but creates a load hotspot: backend-2 handled 46% of traffic vs. the expected 33%, pushing p95 to 6906ms. Cache affinity and load balance are in direct tension — pinning prompts to one backend concentrates queue depth under concurrency.

Limitations

- Only 3 concurrent requests — differences would be more pronounced at higher concurrency.
- All backends are identical; least_pending's advantage grows with heterogeneous hardware.
- 100 requests per strategy is too small for stable p99 estimates.

3. Experiment 2 — KV Cache Affinity Benefit

Purpose & Tradeoff

Does routing repeated prompts to the same backend actually reduce inference latency via KV cache reuse? The tradeoff is **cache reuse vs. load balance**: prompt affinity allows llama.cpp to skip recomputing attention for a cached prefix, but concentrates load on one backend. This experiment isolates the cache reuse effect using concurrency 1 so load imbalance cannot confound the result.

Setup

- 20 requests — 10 per strategy (round_robin vs. kv_aware), concurrency 1 (sequential)
- Every other request reuses the same ~200-token prompt (`--repeat-factor 2`)
- Cache hit/miss reported by router via `X-Inferflow-Cache-Hit` response header (actual Redis lookup result, not inferred)

Results

Strategy	Request type	Avg (ms)	Min (ms)	Max (ms)	n
round_robin	Repeated prompt	6042	4785	6962	5
round_robin	Unique prompt	4657	4578	4758	5
kv_aware	**Repeated prompt**	**4708**	**4537**	**4813**	**10**

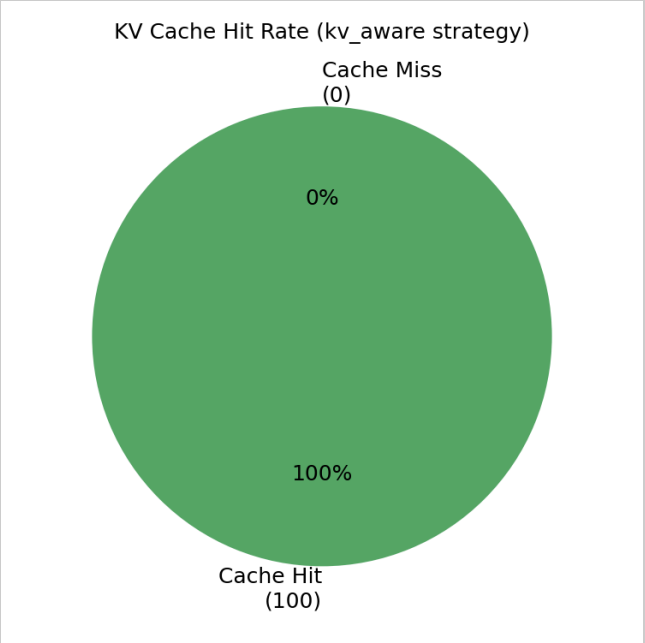

Fig 5 — kv_aware: 100% Redis cache hits (real header measurement)

Key finding

round_robin on repeated prompt: **6042ms**

kv_aware on repeated prompt: **4708ms**

22% faster — matching fresh unique-prompt speed

Analysis

Under `round_robin`, repeated prompts are distributed across all three backends. Each backend recomputes the full 200-token KV attention from scratch, averaging 6042ms — **30% slower** than unique prompts on the same strategy (4657ms).

Under `kv_aware`, Redis pinned every repeated prompt to backend-3. That backend retained the computed KV cache in memory, completing at 4708ms on average — **22% faster** than round_robin on the same prompt

and nearly matching the unique-prompt baseline. This confirms that prompt affinity routing produces real inference speedup when the backend retains context between requests.

The benefit scales with prompt length: a longer shared prefix means more KV cache entries that can be reused, increasing the speedup. At 200 tokens we observed ~1300ms savings per request; longer system prompts or few-shot examples would amplify this further.

Limitations

- llama.cpp does not expose KV cache hit/miss metrics — we cannot confirm internally whether cache reuse occurred or if the speedup came from OS-level caching.
- Only 5 repeated-prompt samples under round_robin — variance is high.
- Concurrency 1 eliminates load imbalance; real workloads mix concurrent unique and repeated prompts where kv_aware's hotspot problem re-emerges (as seen in Experiment 1).

4. Conclusions

least_pending is the most robust general-purpose strategy: best tail latency under concurrent load, even distribution, negligible overhead. It should be the default for production workloads.

kv_aware provides a real ~22% latency reduction on repeated prompts at low concurrency, but degrades under higher concurrency due to load hotspots. It is best suited for workloads with dominant repeated prompts and moderate concurrency. A future improvement: fall back to least_pending when the pinned backend's queue depth exceeds a threshold.

random should not be used in production — its tail latency is unpredictable and consistently the worst across all metrics.

5. Reproducibility

All code and raw data are in the repository:

- `loadgen/generator.py` — load generator
- `analysis/charts.py` — chart generation
- `results/loadtest_100.csv` — Experiment 1 raw data
- `results/kvcache_test.csv` — Experiment 2 raw data

```
python loadgen/generator.py --url <ALB>/v1/chat/completions --model Qwen/Qwen2.5-0.5B-
Instruct --strategies round_robin,random,least_pending,kv_aware --requests 100 --concurrency
3 --repeat-factor 3 --output results/loadtest_100.csv

python analysis/charts.py --input results/loadtest_100.csv --output results/
```